

ApplicationDbContext.cs

Byte 1: The Purpose of ApplicationDbContext

Builds on: None — starting point

In plain terms:

ApplicationDbContext is the bridge between the Minimal API and PostgreSQL. Entity Framework Core (EF Core) uses this class to know **which database entities exist and how they should be stored**.

The code:

```
public class ApplicationDbContext : DbContext
{
    public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
        : base(options)
    {
    }
}
```

What's happening:

The class inherits from EF Core's DbContext. DbContext manages the connection between C# objects and database tables. The DbContextOptions contains configuration such as which database provider and connection string should be used.

Why it matters:

Without this context, EF Core would not have the application's database mapping and configuration needed to communicate with PostgreSQL.

Byte 2: Defining Which Database Table This App Uses

Builds on: Byte 1

In plain terms:

This Minimal API intentionally uses its **own database table** called `minimal_api_applications`. This keeps its data separate from the controller-based application's table.

The code:

```
public const string TableName = "minimal_api_applications";
```

What's happening:

TableName stores the exact PostgreSQL table name used later in the EF Core configuration.

The important distinction is:

Minimal API → minimal_api_applications

Controller API → Applications

Why it matters:

The two implementations can exist in the same database without accidentally reading or modifying each other's application records.

Byte 3: Exposing Applications Through EF Core

Builds on: Byte 1

In plain terms:

DbSet represents a collection of database records that EF Core can query and modify. Here, it exposes the Application model to the database layer.

The code:

```
public DbSet<Models.Application> Applications => Set<Models.Application>();
```

What's happening:

EF Core sees this as the application's database collection. Code elsewhere can perform operations such as:

```
_context.Applications.ToListAsync();
```

or:

```
_context.Applications.Add(application);
```

EF Core translates those operations into SQL.

Why it matters:

This is what allows the API to work with C# Application objects instead of manually writing SQL for every database operation.

Byte 4: Starting Database Model Configuration

Builds on: Bytes 2–3

In plain terms:

OnModelCreating is where we tell EF Core exactly how the C# Application model should map to PostgreSQL.

The code:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);
}
```

```

modelBuilder.Entity<Models.Application>(entity =>
{
    // configuration goes here
});
}

```

What's happening:

EF Core calls this method while building its database model. The modelBuilder lets us configure things such as table names, primary keys, required fields, maximum lengths, and database column types.

Why it matters:

The C# model and database don't automatically have to use identical names or rules. This method explicitly defines the mapping.

Byte 5: Mapping the Entity to the Correct Table

Builds on: Byte 2 and Byte 4

In plain terms:

This is the instruction that makes EF Core store Application objects in `minimal_api_applications` rather than the default table name.

The code:

```

modelBuilder.Entity<Models.Application>(entity =>
{
    entity.ToTable(tableName);
});

```

What's happening:

`ToTable()` maps the Application entity to the table specified by `tableName`.

Because:

`tableName = "minimal_api_applications";`

the mapping becomes:

Application C# entity

↓

`minimal_api_applications` PostgreSQL table

Why it matters:

This is the key separation between the Minimal API database implementation and the controller-based implementation.

Byte 6: Defining the Primary Key

Builds on: Byte 5

In plain terms:

Every application record needs a unique identifier. This configuration tells EF Core that Id is the primary key.

The code:

```
entity.HasKey(a => a.Id);
```

What's happening:

`a => a.Id` means "use the Id property of the Application entity."

EF Core therefore understands:

`Application.Id` → Primary Key

The database can use this value to uniquely identify each application.

Why it matters:

Without a proper key, EF Core cannot reliably track individual records for operations such as updating or deleting them.

Byte 7: Validating the Company, Role, and Status Fields

Builds on: Byte 5

In plain terms:

These properties are required and have maximum lengths. This creates constraints that prevent invalid or excessively long values from being stored.

The code:

```
entity.Property(a => a.Company)
```

```
    .IsRequired()
```

```
    .HasMaxLength(100);
```

```
entity.Property(a => a.Role)
```

```
    .IsRequired()
```

```
    .HasMaxLength(100);
```

```
entity.Property(a => a.Status)
```

```
    .IsRequired()
```

```
.HasMaxLength(20);
```

What's happening:

For example, Company must have a value and can contain at most 100 characters. The same idea applies to Role and Status, with Status limited to 20 characters.

Why it matters:

These rules keep database data consistent and prevent obviously invalid values from being stored.

Byte 8: Configuring Optional Notes

Builds on: Byte 7

In plain terms:

Notes is treated differently from the required fields. It has a size limit, but the code does not mark it as required.

The code:

```
entity.Property(a => a.Notes)
    .HasMaxLength(1000);
```

What's happening:

An application can have no notes, but if notes are provided, they can contain up to 1,000 characters.

Conceptually:

Company → required

Role → required

Status → required

Notes → optional

Why it matters:

This matches the expected behavior of notes: an application does not necessarily need additional comments.

Byte 9: Controlling How Dates Are Stored

Builds on: Byte 8

In plain terms:

The application contains date/time values, and this code explicitly tells PostgreSQL to store them **without timezone information**.

The code:

```
entity.Property(a => a.AppliedOn)
```

```
.HasColumnType("timestamp without time zone");
```

```
entity.Property(a => a.CreatedAt)
```

```
.HasColumnType("timestamp without time zone");
```

What's happening:

Instead of allowing PostgreSQL/Npgsql to infer the column type, the code explicitly selects:

timestamp without time zone

for both date fields.

Why it matters:

It makes the database representation predictable and avoids timezone-kind conflicts when the application works with DateTime values.

Byte 10: The Npgsql DateTime Gotcha

Builds on: Byte 9

In plain terms:

This configuration exists partly because PostgreSQL's timestamp with time zone handling can conflict with certain C# DateTime values. In particular, Npgsql can reject DateTime values whose Kind is Unspecified when they are being written as timestamp with time zone.

The code:

```
// Dates carry no timezone in this app...
```

```
entity.Property(a => a.AppliedOn)
```

```
.HasColumnType("timestamp without time zone");
```

```
entity.Property(a => a.CreatedAt)
```

```
.HasColumnType("timestamp without time zone");
```

What's happening:

The application deliberately chooses a database type that does not require timezone semantics. That matches the application's intended handling of these values.

Why it matters:

Without this explicit mapping, the application could encounter runtime database errors when EF Core/Npgsql tries to save an incompatible DateTime.

Byte 11: The Complete Mapping Pipeline

Builds on: All previous bytes

In plain terms:

The entire class establishes one clear pipeline: the Minimal API works with C# Application objects, EF Core maps them to a specific PostgreSQL table, and the configuration defines the rules for each database field.

The code:

```
public DbSet<Models.Application> Applications
    => Set<Models.Application>();

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    modelBuilder.Entity<Models.Application>(entity =>
    {
        entity.ToTable(TableName);
        entity.HasKey(a => a.Id);

        entity.Property(a => a.Company)
            .IsRequired()
            .HasMaxLength(100);

        entity.Property(a => a.Role)
            .IsRequired()
            .HasMaxLength(100);

        entity.Property(a => a.Status)
            .IsRequired()
            .HasMaxLength(20);

        entity.Property(a => a.Notes)
            .HasMaxLength(1000);
    });
}
```

```

        entity.Property(a => a.AppliedOn)
            .HasColumnType("timestamp without time zone");

        entity.Property(a => a.CreatedAt)
            .HasColumnType("timestamp without time zone");
    });
}

```

What's happening:

DbSet exposes the entity to the application, while OnModelCreating defines how that entity is represented in PostgreSQL. Together, they form the EF Core database mapping for the Minimal API.

Why it matters:

If another developer needs to understand where the Minimal API's application data goes and what database rules apply to it, this class is the central place to look.

PUTTING IT TOGETHER

Think of ApplicationDbContext as the **database rulebook for the Minimal API**. The DbSet says, "this application works with Application records," while ToTable() says those records belong in minimal_api_applications. The remaining configuration defines the database rules: Id is the key, some fields are required, fields have length limits, and dates use timestamp without time zone. When an endpoint asks EF Core to create, read, update, or delete an Application, EF Core uses these mappings to translate between the C# object and the PostgreSQL table.

ApplicationDtos.cs

Byte 1: Why These DTOs Exist

Builds on: None — starting point

In plain terms:

A **DTO (Data Transfer Object)** is a class used to move data between parts of an application without directly exposing the database entity. This file defines three DTO responsibilities: creating an application, updating an application, and sending an application back to the client.

The code:

```
public class CreateApplicationDto : IValidatableObject
{
    // data coming from the client
}
```

```
public class UpdateApplicationDto : IValidatableObject
{
    // data coming from the client
}
```

```
public class ApplicationResponseDto
{
    // data going back to the client
}
```

What's happening:

The first two DTOs represent **incoming API data**. The response DTO represents **outgoing API data**.

Why it matters:

This separates the API contract from the database model. Changes to the database entity don't automatically mean the API has to expose those changes.

Byte 2: Centralizing Allowed Status Values

Builds on: Byte 1

In plain terms:

The application has a fixed set of valid application statuses. Instead of defining those values in multiple places, `ApplicationStatus` provides one shared list.

The code:

```
public static class ApplicationStatus
{
    public static readonly string[] Allowed =
    {
        "Wishlist",
        "Applied",
        "Interview",
        "Offer",
        "Rejected"
    };
}
```

What's happening:

Because the class is static, it doesn't need to be instantiated. Any code can access:

`ApplicationStatus.Allowed`

and check whether a status exists in the allowed list.

Why it matters:

Centralizing the list reduces the chance that different parts of the application accidentally use different status values.

Byte 3: The Create DTO Defines Incoming Data

Builds on: Byte 1

In plain terms:

`CreateApplicationDto` describes the data the client is allowed to send when creating a new application.

The code:

```
public class CreateApplicationDto : IValidatableObject
{
    public string Company { get; set; } = string.Empty;
    public string Role { get; set; } = string.Empty;
```

```
public string Status { get; set; } = "Applied";  
public DateTime? AppliedOn { get; set; }  
public string? JobUrl { get; set; }  
public string? Notes { get; set; }  
}
```

What's happening:

The DTO contains only the fields needed from the client. Notice that fields such as Id and CreatedAt are absent because those are normally controlled by the server/database.

Why it matters:

The client doesn't get to decide server-controlled values such as the database ID or creation timestamp.

Byte 4: Required and Length Validation

Builds on: Byte 3

In plain terms:

Data annotations provide automatic validation rules for fields such as Company and Role.

The code:

```
[Required(ErrorMessage = "Company is required.")]  
[StringLength(100, MinimumLength = 2,  
    ErrorMessage = "Company must be 2-100 characters.")]  
public string Company { get; set; } = string.Empty;
```

What's happening:

[Required] means the property must have a value. [StringLength] restricts the length to between 2 and 100 characters.

The same pattern is used for Role.

Why it matters:

Basic invalid input can be rejected before it reaches the database.

Byte 5: Defaulting the Status

Builds on: Byte 3

In plain terms:

If a new application doesn't explicitly provide a status, the DTO starts with "Applied" as its default value.

The code:

```
[Required(ErrorMessage = "Status is required.")]
```

```
public string Status { get; set; } = "Applied";
```

What's happening:

When the DTO is created, Status initially contains "Applied".

This is a **default value**, not the complete validation rule. The later Validate() method still checks whether the final status is actually one of the allowed values.

Why it matters:

It gives new applications a sensible starting status while still preventing unsupported status values.

Byte 6: Nullable Fields Represent Optional Information

Builds on: Byte 3

In plain terms:

Some application information isn't mandatory, so the DTO allows it to be absent.

The code:

```
public DateTime? AppliedOn { get; set; }
```

```
public string? JobUrl { get; set; }
```

```
public string? Notes { get; set; }
```

What's happening:

The ? means the value can be null.

For example:

JobUrl → may be missing

Notes → may be missing

AppliedOn → may be missing in some situations

AppliedOn is especially interesting because the later custom validation decides **when** it is actually required.

Why it matters:

Not every application needs every piece of information. Making these properties nullable allows the DTO to represent those legitimate cases.

Byte 7: URL and Notes Validation

Builds on: Byte 4

In plain terms:

The DTO also validates the format and size of optional fields.

The code:

```
[Url(ErrorMessage =  
    "Please enter a valid URL (e.g. https://example.com/job).")]  
public string? JobUrl { get; set; }  
  
[StringLength(1000,  
    ErrorMessage = "Notes cannot be longer than 1000 characters.")]  
public string? Notes { get; set; }
```

What's happening:

[Url] tells the validation system to check that JobUrl looks like a valid URL.
[StringLength(1000)] limits notes to 1,000 characters.

Why it matters:

These rules prevent malformed URLs and excessively large notes from entering the application through the API.

Byte 8: IValidatableObject Enables Business Rules

Builds on: Bytes 4–7

In plain terms:

Data annotations handle simple field-level rules, but some rules depend on **multiple fields**. IValidatableObject allows the DTO to implement those custom business rules.

The code:

```
public class CreateApplicationDto : IValidatableObject  
{  
    public IEnumerable<ValidationResult> Validate(  
        ValidationContext validationContext)  
    {  
        // custom validation rules  
    }  
}
```

What's happening:

The framework can call Validate() and collect any ValidationResult objects returned by it.

This is useful for rules such as:

"If Status is not Wishlist, AppliedOn must exist."

That rule cannot be expressed cleanly using [Required] on AppliedOn, because AppliedOn is only conditionally required.

Why it matters:

It keeps cross-field/business validation inside the DTO rather than scattering it across API endpoints.

Byte 9: Validating the Application Status

Builds on: Bytes 2 and 8

In plain terms:

The custom validator checks that the supplied status belongs to the application's official status list.

The code:

```
if (!ApplicationStatus.Allowed.Contains(Status))
{
    yield return new ValidationResult(
        "Status must be one of: Wishlist, Applied, Interview, Offer, Rejected.",
        new[] { nameof(Status) });
}
```

What's happening:

Contains(Status) searches the centralized Allowed array.

If the status is something like:

"Pending"

the validation fails and associates the error with the Status property.

Why it matters:

The API won't accept arbitrary status strings that the rest of the application doesn't understand.

Byte 10: Extra Protection for Company and Role

Builds on: Byte 4 and Byte 8

In plain terms:

The custom validation performs an additional check for blank or whitespace-only company and role names.

The code:

```
if (string.IsNullOrWhiteSpace(Company) || Company.Trim().Length < 2)
```

```
{
  yield return new ValidationResult(
    "Company must be 2-100 characters.",
    new[] { nameof(Company) });
}
```

What's happening:

This catches values such as:

""

" "

"A"

[StringLength] handles length constraints, while this custom check explicitly rejects whitespace-only values and checks the trimmed length.

The same logic exists for Role.

Why it matters:

A string containing spaces can technically have characters while still being meaningless input. This provides stronger validation than relying only on the attribute.

Byte 11: Conditional AppliedOn Validation

Builds on: Bytes 5 and 8

In plain terms:

The application date is required for most statuses, but not when the application is still in the Wishlist.

The code:

```
if (Status != "Wishlist" && AppliedOn is null)
{
  yield return new ValidationResult(
    "Applied date is required unless status is Wishlist.",
    new[] { nameof(AppliedOn) });
}
```

What's happening:

The rule is:

Wishlist + no AppliedOn → valid

Applied + no AppliedOn → invalid

Interview + no AppliedOn → invalid

Offer + no AppliedOn → invalid

Rejected + no AppliedOn → invalid

This is a classic **conditional validation** rule because the requirement for one property depends on another property's value.

Why it matters:

A job that is only being considered hasn't necessarily been applied to yet, so requiring an application date for Wishlist would be incorrect.

Byte 12: Update DTO Mirrors Create Validation

Builds on: Bytes 3–11

In plain terms:

UpdateApplicationDto uses essentially the same fields and validation rules as CreateApplicationDto, but its purpose is updating an existing application.

The code:

```
public class UpdateApplicationDto : IValidatableObject
{
    [Required]
    [StringLength(100, MinimumLength = 2)]
    public string Company { get; set; } = string.Empty;

    [Required]
    [StringLength(100, MinimumLength = 2)]
    public string Role { get; set; } = string.Empty;

    public string Status { get; set; } = "Applied";

    // ...
}
```

What's happening:

The update request is validated using the same concepts: required fields, length limits, valid statuses, valid URLs, notes length, and the conditional AppliedOn rule.

Why it matters:

An existing application should follow the same data rules when modified. The tradeoff is that validation logic is duplicated between the create and update DTOs.

Byte 13: The Response DTO Defines What Leaves the API

Builds on: Bytes 1–3

In plain terms:

`ApplicationResponseDto` is the opposite direction from the create/update DTOs. It describes the data the server sends back to the client.

The code:

```
public class ApplicationResponseDto
{
    public int Id { get; set; }
    public string Company { get; set; } = string.Empty;
    public string Role { get; set; } = string.Empty;
    public string Status { get; set; } = string.Empty;
    public DateTime? AppliedOn { get; set; }
    public string? JobUrl { get; set; }
    public string? Notes { get; set; }
    public DateTime CreatedAt { get; set; }
}
```

What's happening:

Unlike the create DTO, this includes server-generated information such as `Id` and `CreatedAt`.

The direction is therefore:

Client → Create/Update DTO → Server

Server → Response DTO → Client

Why it matters:

It gives the API explicit control over what information is exposed to the frontend.

Byte 14: Converting an Entity into a Response DTO

Builds on: Byte 13

In plain terms:

FromEntity() converts the database Application object into the DTO that the API can safely return.

The code:

```
public static ApplicationResponseDto FromEntity(
    Models.Application app) => new()
{
    Id = app.Id,
    Company = app.Company,
    Role = app.Role,
    Status = app.Status,
    AppliedOn = app.AppliedOn,
    JobUrl = app.JobUrl,
    Notes = app.Notes,
    CreatedAt = app.CreatedAt
};
```

What's happening:

The method receives an Application entity and creates a new ApplicationResponseDto, copying the required values across.

This is called **mapping**: converting one object representation into another.

Why it matters:

The API doesn't have to return the EF Core entity directly. It explicitly chooses which properties are exposed in the response.

Byte 15: The Complete Data Flow

Builds on: All previous bytes

In plain terms:

The file creates a boundary between client data, validation, database entities, and API responses.

The code:

Client

↓

CreateApplicationDto / UpdateApplicationDto

↓
Validation
↓
Models.Application
↓
EF Core / PostgreSQL

PostgreSQL
↓
Models.Application
↓
ApplicationResponseDto.FromEntity()
↓
Client

What's happening:

Incoming data is represented by DTOs and validated before being used by the application. Database entities represent stored data. When information needs to go back to the frontend, it is mapped into the response DTO.

Why it matters:

This separation makes the API easier to control and maintain: **input validation, database representation, and output representation are not all mixed into one class.**

PUTTING IT TOGETHER

This file defines the **API's data-transfer and validation layer**. ApplicationStatus centralizes the valid status values, while the create and update DTOs define what the client can send and validate both simple rules and business rules. ApplicationResponseDto defines what the server sends back, and FromEntity() performs the conversion from the EF Core entity to that response shape. So the overall flow is **request → DTO → validation → entity → database**, and then **database → entity → response DTO → client**.

ApplicationEndpoints.cs

Byte 1: What ApplicationEndpoints Is Responsible For

Builds on: None — starting point

In plain terms:

ApplicationEndpoints contains all the HTTP endpoints for the Placement Tracker's Minimal API. It handles the five basic CRUD operations: **list, get one, create, update, and delete** an application.

The code:

```
public static class ApplicationEndpoints
{
    public static void MapApplicationEndpoints(this WebApplication app)
    {
        var group = app.MapGroup("/api/applications");

        // endpoints go here
    }
}
```

What's happening:

This is a static class because it mainly provides endpoint-mapping logic. MapApplicationEndpoints() is called during application startup to register the API routes.

Why it matters:

This replaces the traditional controller approach. Instead of creating an ApplicationsController with [HttpGet], [HttpPost], etc., Minimal API defines the routes directly with MapGet, MapPost, MapPut, and MapDelete.

Byte 2: Creating a Common Route Group

Builds on: Byte 1

In plain terms:

All application-related endpoints share the same base URL: /api/applications. The route group prevents us from repeating that path for every endpoint.

The code:

```
var group = app.MapGroup("/api/applications");
```

What's happening:

Once the group exists, routes defined on group automatically start with:

/api/applications

So:

```
group.MapGet("/");
```

becomes:

GET /api/applications

Why it matters:

It keeps the endpoint definitions shorter and makes the API structure easier to see.

Byte 3: GET — Retrieve All Applications

Builds on: Bytes 1–2

In plain terms:

The first endpoint handles a request for the complete application list. It gets records from EF Core, sorts newest IDs first, converts them to response DTOs, and returns HTTP 200.

The code:

```
group.MapGet("/", async (ApplicationDbContext db) =>
{
    var apps = await db.Applications
        .OrderByDescending(a => a.Id)
        .ToListAsync();

    return Results.Ok(
        apps.Select(ApplicationResponseDto.FromEntity).ToList());
});
```

What's happening:

The ApplicationDbContext is injected directly into the endpoint. EF Core queries the database, retrieves the applications, and orders them by descending Id.

The results are then mapped from database entities to ApplicationResponseDto.

Why it matters:

This is the basic **Read** part of CRUD and provides the frontend with the current application list.

Byte 4: GET — Retrieve One Application

Builds on: Byte 3

In plain terms:

This endpoint retrieves one application based on its ID. If the ID doesn't exist, it returns 404 Not Found.

The code:

```
group.MapGet("/{id:int}", async (int id, ApplicationDbContext db) =>
{
    var app = await db.Applications.FindAsync(id);

    if (app is null)
    {
        return Results.NotFound(
            new { message = $"Application {id} not found." });
    }

    return Results.Ok(
        ApplicationResponseDto.FromEntity(app));
});
```

What's happening:

{id:int} means the route expects an integer ID:

GET /api/applications/5

FindAsync(id) searches for that database record. If it exists, the endpoint returns it with 200 OK; otherwise it returns 404.

Why it matters:

The frontend needs a way to retrieve a specific application rather than downloading the entire collection.

Byte 5: POST — Start Creating an Application

Builds on: Bytes 1–4

In plain terms:

The POST endpoint handles creation of a new application. The incoming request is automatically bound to CreateApplicationDto, then validated before anything is saved.

The code:

```
group.MapPost("/", async (
    CreateApplicationDto dto,
    ApplicationDbContext db) =>
{
    var errors = ValidateDto(dto);

    if (errors is not null)
    {
        return errors;
    }

    // creation continues...
});
```

What's happening:

The JSON request body is converted into a CreateApplicationDto.

For example:

```
{
  "company": "Google",
  "role": "ML Engineer",
  "status": "Applied"
}
```

The endpoint then calls the custom ValidateDto() method.

Why it matters:

Validation happens **before** the application is converted into a database entity, preventing invalid input from reaching the database.

Byte 6: Converting the DTO into a Database Entity

Builds on: Byte 5

In plain terms:

After validation succeeds, the endpoint creates the actual Application entity that EF Core will store in PostgreSQL.

The code:

```
var app = new Models.Application
{
    Company = dto.Company.Trim(),
    Role = dto.Role.Trim(),
    Status = dto.Status,
    AppliedOn = dto.AppliedOn,
    JobUrl = NormalizeOptional(dto.JobUrl),
    Notes = NormalizeOptional(dto.Notes),
    CreatedAt = DateTime.UtcNow
};
```

What's happening:

The endpoint maps values from the incoming DTO into the database model.

It also performs some cleanup:

- Trim() removes unnecessary spaces.
- NormalizeOptional() converts empty optional values to null.
- CreatedAt is generated by the server rather than trusting the client.

Why it matters:

The DTO represents **what the client sends**, while the entity represents **what the database stores**. This is the point where the two are connected.

Byte 7: Saving the New Application

Builds on: Byte 6

In plain terms:

Once the entity is prepared, EF Core tracks it and saves it to PostgreSQL.

The code:

```
db.Applications.Add(app);
await db.SaveChangesAsync();
```

What's happening:

Add() tells EF Core that this is a new database record.

SaveChangesAsync() actually sends the required SQL operation to the database.

After saving, EF Core can populate generated values such as the new Id.

Why it matters:

Creating the C# object doesn't automatically create a database row. `SaveChangesAsync()` is the point where the database operation is committed.

Byte 8: POST Returns 201 Created

Builds on: Byte 7

In plain terms:

After successfully creating the record, the API returns 201 Created along with the newly created application.

The code:

```
return Results.Created(  
    $"/api/applications/{app.Id}",  
    ApplicationResponseDto.FromEntity(app));
```

What's happening:

`Results.Created()` produces an HTTP 201 response.

The first argument tells the client where the new resource can be found:

`/api/applications/15`

The second argument contains the created application.

Why it matters:

201 Created communicates more precisely than 200 OK that a new resource was successfully created.

Byte 9: PUT — Find the Application to Update

Builds on: Bytes 4–7

In plain terms:

The PUT endpoint updates an existing application. Before changing anything, it validates the incoming DTO and looks up the existing database record.

The code:

```
group.MapPut("/{id:int}", async (  
    int id,  
    UpdateApplicationDto dto,  
    ApplicationDbContext db) =>  
{  
    var errors = ValidateDto(dto);
```

```

    if (errors is not null)
    {
        return errors;
    }

    var existing = await db.Applications.FindAsync(id);

    if (existing is null)
    {
        return Results.NotFound(
            new { message = $"Application {id} not found." });
    }

    // update continues...
});

```

What's happening:

There are two separate checks:

Invalid request → 400

Valid request but ID doesn't exist → 404

Only after both checks pass does the update happen.

Why it matters:

This prevents the API from attempting to modify a record that doesn't exist.

Byte 10: PUT — Modify the Existing Entity

Builds on: Byte 9

In plain terms:

Instead of creating a new object, PUT changes the properties of the entity that EF Core already retrieved.

The code:

```

existing.Company = dto.Company.Trim();
existing.Role = dto.Role.Trim();

```

```
existing.Status = dto.Status;  
existing.AppliedOn = dto.AppliedOn;  
existing.JobUrl = NormalizeOptional(dto.JobUrl);  
existing.Notes = NormalizeOptional(dto.Notes);
```

What's happening:

EF Core is already tracking existing. When its properties change, EF Core knows that the entity has been modified.

The code deliberately doesn't replace existing with a completely new object.

Why it matters:

EF Core can detect these changes and generate the appropriate SQL UPDATE when `SaveChangesAsync()` is called.

Byte 11: PUT — Save and Return the Updated Record

Builds on: Byte 10

In plain terms:

After modifying the tracked entity, the endpoint saves the changes and returns the updated application.

The code:

```
await db.SaveChangesAsync();  
  
return Results.Ok(  
    ApplicationResponseDto.FromEntity(existing));
```

What's happening:

`SaveChangesAsync()` sends the update to PostgreSQL. The endpoint then returns the updated object with 200 OK.

Why it matters:

The frontend immediately receives the final server-side version of the application after the update.

Byte 12: DELETE — Find the Record First

Builds on: Byte 4

In plain terms:

DELETE first looks up the application by ID. If there is nothing to delete, it returns 404.

The code:

```
var app = await db.Applications.FindAsync(id);

if (app is null)
{
    return Results.NotFound(
        new { message = $"Application {id} not found." });
}
```

What's happening:

This follows the same lookup pattern used by GET and PUT.

The endpoint doesn't blindly attempt to delete an unknown ID.

Why it matters:

It gives the client a meaningful response when the requested application doesn't exist.

Byte 13: DELETE — Remove and Return 204

Builds on: Byte 12

In plain terms:

Once the application is found, it is marked for deletion, saved to PostgreSQL, and the API returns 204 No Content.

The code:

```
db.Applications.Remove(app);
await db.SaveChangesAsync();
```

```
return Results.NoContent();
```

What's happening:

Remove() tells EF Core that the entity should be deleted. SaveChangesAsync() executes the deletion in the database.

204 No Content means the operation succeeded but there is no response body to return.

Why it matters:

DELETE doesn't need to send the deleted application back to the client, so 204 is appropriate.

Byte 14: Normalizing Optional Values

Builds on: Byte 6

In plain terms:

NormalizeOptional() cleans up optional strings. An empty or whitespace-only value becomes null; otherwise, the surrounding spaces are removed.

The code:

```
private static string? NormalizeOptional(string? value) =>
    string.IsNullOrEmpty(value) ? null : value.Trim();
```

What's happening:

For example:

" https://example.com " → "https://example.com"

" " → null

"" → null

null → null

Why it matters:

Without normalization, the database could contain meaningless empty strings where null would better represent "no value."

Byte 15: Why Minimal APIs Need Manual Validation Here

Builds on: Byte 5

In plain terms:

The DTOs use DataAnnotations such as [Required] and [StringLength], but this Minimal API explicitly calls ValidateDto() because it isn't relying on controller-style automatic ModelState validation.

The code:

```
var errors = ValidateDto(dto);
```

```
if (errors is not null)
```

```
{
    return errors;
}
```

What's happening:

ValidateDto() manually invokes .NET's validation system and converts the validation results into an HTTP validation response.

Why it matters:

Without this explicit validation call, the endpoint would not automatically execute these DTO validation rules in the same way a controller using [ApiController] typically does.

Byte 16: Creating the Validation Context

Builds on: Byte 15

In plain terms:

ValidationContext provides the validation system with the object it needs to validate.

The code:

```
var context = new ValidationContext(dto);  
var results = new List<ValidationResult>();
```

```
Validator.TryValidateObject(  
    dto,  
    context,  
    results,  
    validateAllProperties: true);
```

What's happening:

TryValidateObject() examines the DTO's validation attributes and its IValidatableObject.Validate() implementation.

For example, it can detect:

Company is missing

Role is too short

Status is invalid

AppliedOn is missing for a non-Wishlist status

The discovered errors are placed into results.

Why it matters:

This is the bridge between the validation rules defined in the DTOs and the HTTP response returned by the endpoint.

Byte 17: Turning Validation Results into API Errors

Builds on: Byte 16

In plain terms:

The raw validation results aren't returned directly. The method reorganizes them into a dictionary where each property name points to one or more error messages.

The code:

```
var errors = new Dictionary<string, string[]>();
```

```
foreach (var result in results)
```

```
{
```

```
    var keys = result.MemberNames.Any()
```

```
        ? result.MemberNames
```

```
        : new[] { string.Empty };
```

```
    foreach (var key in keys)
```

```
    {
```

```
        var name = string.IsNullOrEmpty(key)
```

```
            ? "Error"
```

```
            : key;
```

```
        // add the error message to errors[name]
```

```
    }
```

```
}
```

What's happening:

The method groups validation messages by field.

Conceptually, the result can look like:

```
{
```

```
    "Company": [
```

```
        "Company must be 2-100 characters."
```

```
    ],
```

```
    "Status": [
```

```
        "Status must be one of: ..."
```

```
    ]
```

```
}
```

Why it matters:

This gives the frontend a structured response that can associate each error with the corresponding form field.

Byte 18: Supporting Multiple Errors for One Field

Builds on: Byte 17

In plain terms:

A single field can potentially produce multiple validation errors, so the method appends new messages rather than overwriting previous ones.

The code:

```
if (errors.TryGetValue(name, out var existing))
{
    errors[name] = existing
        .Append(result.ErrorMessage ?? "Invalid value.")
        .ToArray();
}
else
{
    errors[name] = new[]
    {
        result.ErrorMessage ?? "Invalid value."
    };
}
```

What's happening:

If Company already has an error, another validation message is added to its array.

So the structure supports:

Company → [error 1, error 2, ...]

Status → [error 1, ...]

Why it matters:

It prevents one validation error from accidentally hiding another validation error for the same property.

Byte 19: Returning the Final Validation Response

Builds on: Bytes 17–18

In plain terms:

Once all validation errors have been organized, `Results.ValidationProblem()` turns them into an HTTP 400 validation response.

The code:

```
return Results.ValidationProblem(errors);
```

And when there are no errors:

```
if (results.Count == 0)
```

```
{
```

```
    return null;
```

```
}
```

What's happening:

The method has two possible outcomes:

Validation succeeds → null

Validation fails → HTTP 400 validation response

The endpoint checks for null before continuing.

Why it matters:

This creates a consistent validation flow across both POST and PUT without duplicating validation logic inside each endpoint.

Byte 20: The Full CRUD Request Flow

Builds on: All previous bytes

In plain terms:

The entire file follows the same general pattern: receive a request, validate it when necessary, interact with EF Core, and return an appropriate HTTP result.

The code:

GET → Query database → Return DTOs

GET /id → Find record → 200 or 404

POST → Validate DTO → Create entity

→ Save → 201 Created

PUT → Validate DTO → Find entity

→ Modify → Save → 200 OK

DELETE → Find entity → Remove → Save
→ 204 No Content

What's happening:

The endpoints are thin: they coordinate HTTP requests, validation, DTO/entity mapping, and database operations. `ApplicationDbContext` handles database communication, while the DTOs handle input/output shapes and validation.

Why it matters:

Once you understand this flow, the whole Minimal API becomes much easier to follow: **route** → **validate** → **database operation** → **response**.

PUTTING IT TOGETHER

`ApplicationEndpoints` is the HTTP layer of the Minimal API. `MapGroup()` establishes `/api/applications`, then `GET`, `POST`, `PUT`, and `DELETE` implement the CRUD operations. `POST` and `PUT` receive DTOs, manually validate them because this Minimal API is not using controller-style automatic validation, and then interact with `ApplicationDbContext`. The response DTO keeps database entities separate from what the API exposes to the frontend. Finally, `NormalizeOptional()` and `ValidateDto()` handle input cleanup and consistent validation errors, giving the whole API a predictable request → validation → database → response flow.

Application.cs

Byte 1: What the Application Model Represents

Builds on: None — starting point

In plain terms:

Application is the main **model/entity** representing one job or internship application in the system. One C# object corresponds to one row in the `minimal_api_applications` PostgreSQL table.

The code:

```
public class Application : IValidatableObject
{
    public int Id { get; set; }

    public string Company { get; set; } = string.Empty;
    public string Role { get; set; } = string.Empty;
    public string Status { get; set; } = "Applied";

    // other properties...
}
```

What's happening:

The class describes the data belonging to an application: company, role, status, dates, URL, notes, etc. It also implements `IValidatableObject`, which allows the model to contain custom validation rules.

Why it matters:

This is the central object shared by the database layer and application logic. EF Core uses it to map C# objects to PostgreSQL records.

Byte 2: The Id Identifies Each Application

Builds on: Byte 1

In plain terms:

Every application needs a unique identifier so the API can tell records apart. Id is the property's value used as the database primary key through the EF Core configuration.

The code:

```
public int Id { get; set; }
```

What's happening:

For example:

Application 1 → Id = 1

Application 2 → Id = 2

Application 3 → Id = 3

The endpoints can then use the ID in URLs such as:

GET /api/applications/2

Why it matters:

Without a unique identifier, operations such as GET-by-ID, UPDATE, and DELETE couldn't reliably target a specific application.

Byte 3: Company and Role Define the Job

Builds on: Byte 1

In plain terms:

Company and Role store the basic identity of the opportunity. Both are required and must contain between 2 and 100 characters.

The code:

```
[Required(ErrorMessage = "Company is required.")]
```

```
[StringLength(100, MinimumLength = 2,
```

```
    ErrorMessage = "Company must be 2-100 characters.")]
```

```
public string Company { get; set; } = string.Empty;
```

```
[Required(ErrorMessage = "Role is required.")]
```

```
[StringLength(100, MinimumLength = 2,
```

```
    ErrorMessage = "Role must be 2-100 characters.")]
```

```
public string Role { get; set; } = string.Empty;
```

What's happening:

[Required] prevents the property from being missing, while [StringLength] restricts its size.

For example:

"Microsoft" → valid

"ML Engineer" → valid

"A" → invalid

Why it matters:

These rules prevent incomplete or obviously invalid application records from entering the system.

Byte 4: Status Tracks the Application's Progress

Builds on: Byte 1

In plain terms:

Status describes where the application currently stands in the job-search process. It defaults to "Applied".

The code:

```
[Required(ErrorMessage = "Status is required.")]  
public string Status { get; set; } = "Applied";
```

What's happening:

If a new Application object doesn't explicitly set Status, it starts as:

Applied

However, [Required] alone doesn't restrict which string can be used. That restriction is handled separately by AllowedStatuses and Validate().

Why it matters:

Separating the default value from the allowed-value validation makes it clear that "Applied" is simply the default, while the system still supports the other valid statuses.

Byte 5: Centralizing the Valid Statuses

Builds on: Byte 4

In plain terms:

AllowedStatuses defines the complete set of status values that the application considers valid.

The code:

```
public static readonly string[] AllowedStatuses =  
    new[]  
    {  
        "Wishlist",  
        "Applied",  
        "Interview",  
        "Offer",  
        "Rejected"
```

```
};
```

What's happening:

The validation logic can check whether the current status exists in this array.

Conceptually:

Wishlist ✓

Applied ✓

Interview ✓

Offer ✓

Rejected ✓

Something ✗

Why it matters:

It prevents arbitrary status strings from being stored and gives the application a single source for the allowed values.

Byte 6: AppliedOn Represents When the Application Was Submitted

Builds on: Bytes 4–5

In plain terms:

AppliedOn stores the date on which the job application was submitted. It is nullable because a wishlist item hasn't necessarily been applied to yet.

The code:

```
[DataType(DataType.Date)]
```

```
[Display(Name = "Applied On")]
```

```
public DateTime? AppliedOn { get; set; }
```

What's happening:

DateTime? means the property can contain either a date/time value or null.

The [DataType(DataType.Date)] attribute describes it as a date-oriented field, while [Display] provides a friendly display name.

Why it matters:

A Wishlist application can legitimately have no application date. The custom validation later enforces when the date becomes mandatory.

Byte 7: Storing an Optional Job URL

Builds on: Byte 1

In plain terms:

JobUrl stores a link to the job posting, but the link is optional.

The code:

```
[Url(ErrorMessage =  
    "Please enter a valid URL (e.g. https://example.com/job).")]  
[Display(Name = "Job URL")]  
public string? JobUrl { get; set; }
```

What's happening:

The ? allows null, while [Url] asks the validation system to check that a supplied value looks like a URL.

[Display] gives UI code a human-friendly label: "Job URL".

Why it matters:

The model can represent applications where no job-posting link was saved while still validating links when they are provided.

Byte 8: Optional Notes

Builds on: Byte 1

In plain terms:

Notes allows extra information about an application, such as interview details or reminders. It is optional but limited to 1,000 characters.

The code:

```
[StringLength(  
    1000,  
    ErrorMessage = "Notes cannot be longer than 1000 characters.")]  
public string? Notes { get; set; }
```

What's happening:

string? allows no notes at all, while [StringLength] prevents excessively long notes.

Why it matters:

It gives users flexibility to store additional information without allowing unlimited text in this field.

Byte 9: Automatically Setting CreatedAt

Builds on: Byte 1

In plain terms:

CreatedAt records when the application object was created. If no value is supplied, it automatically uses the current UTC time.

The code:

```
[Display(Name = "Created At")]
```

```
public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
```

What's happening:

When a new Application object is instantiated, the property gets a timestamp such as:

2026-09-25 13:20:00 UTC

The API's POST endpoint can therefore create the timestamp server-side.

Why it matters:

The creation time should generally come from the server rather than trusting the client to provide a correct timestamp.

Byte 10: Why IValidatableObject Is Used

Builds on: Bytes 3–9

In plain terms:

Attributes such as [Required] and [StringLength] handle simple rules for individual properties. IValidatableObject is used when a rule depends on **multiple properties or custom logic**.

The code:

```
public class Application : IValidatableObject
{
    public IEnumerable<ValidationResult> Validate(
        ValidationContext validationContext)
    {
        // custom business validation
    }
}
```

What's happening:

The class promises to provide a Validate() method. .NET's validation system can call this method and collect any ValidationResult objects it produces.

Why it matters:

Rules like "AppliedOn is required unless Status is Wishlist" cannot be expressed simply with [Required] on AppliedOn.

Byte 11: Validating the Status Value

Builds on: Bytes 5 and 10

In plain terms:

The custom validator checks that the current status is one of the predefined allowed statuses.

The code:

```
if (!AllowedStatuses.Contains(Status))
{
    yield return new ValidationResult(
        "Status must be one of: Wishlist, Applied, Interview, Offer, Rejected.",
        new[] { nameof(Status) });
}
```

What's happening:

Contains() searches AllowedStatuses.

If Status is "Pending", it isn't found, so a ValidationResult is returned and associated with the Status property.

Why it matters:

This is what actually enforces the allowed-status list. The [Required] attribute only checks that some value exists.

Byte 12: Rejecting Blank Company Names

Builds on: Bytes 3 and 10

In plain terms:

The custom validation adds another layer of protection for Company: it rejects empty, whitespace-only, or effectively too-short values.

The code:

```
if (string.IsNullOrEmpty(Company)
    || Company.Trim().Length < 2)
{
    yield return new ValidationResult(
        "Company must be 2-100 characters.",
        new[] { nameof(Company) });
}
```

What's happening:

This catches values such as:

""

" "

"A"

Trim() removes surrounding whitespace before checking the meaningful length.

Why it matters:

A value containing only spaces technically contains characters, but it isn't a meaningful company name. This custom rule closes that gap.

Byte 13: Rejecting Blank Role Names

Builds on: Byte 12

In plain terms:

The same business rule is applied to Role: it must contain at least two meaningful characters.

The code:

```
if (string.IsNullOrEmpty(Role)
    || Role.Trim().Length < 2)
{
    yield return new ValidationResult(
        "Role must be 2-100 characters.",
        new[] { nameof(Role) });
}
```

What's happening:

The validator rejects empty, whitespace-only, or one-character roles.

Why it matters:

This keeps Role consistent with the validation rules already applied to Company.

Byte 14: Conditional Validation of AppliedOn

Builds on: Bytes 6, 10, and 11

In plain terms:

The application date is required for every status except Wishlist. This is a **conditional business rule** because whether the field is required depends on another field.

The code:

```

if (Status != "Wishlist" && AppliedOn is null)
{
    yield return new ValidationResult(
        "Applied date is required unless status is Wishlist.",
        new[] { nameof(AppliedOn) });
}

```

What's happening:

The logic is essentially:

Wishlist + no date → valid

Applied + no date → invalid

Interview + no date → invalid

Offer + no date → invalid

Rejected + no date → invalid

Why it matters:

A wishlist entry represents an opportunity the user hasn't necessarily applied to, so requiring an application date in that state would be incorrect.

Byte 15: How the Model Combines Attribute and Business Validation

Builds on: All previous bytes

In plain terms:

This model uses **two levels of validation**: attributes handle straightforward field rules, while `Validate()` handles rules involving relationships between fields or additional business logic.

The code:

```

[Required]
[StringLength(100, MinimumLength = 2)]
public string Company { get; set; }

public IEnumerable<ValidationResult> Validate(
    ValidationContext validationContext)
{
    if (Status != "Wishlist" && AppliedOn is null)
    {

```

```
        // business rule
    }
}
```

What's happening:

DataAnnotations



Simple field rules



Required / Length / URL

InvalidatableObject



Business rules



Allowed status / conditional date / whitespace checks

The endpoint's validation process can evaluate both types.

Why it matters:

This keeps simple validation declarative and puts more complicated application-specific rules in one explicit place.

PUTTING IT TOGETHER

Application is the central data model representing one placement-tracker record. Its properties describe the application's data, while DataAnnotations define straightforward validation such as required fields, lengths, and URL format. AllowedStatuses defines the valid lifecycle states, and Validate() adds the rules that require actual business logic, such as requiring AppliedOn unless the application is still in Wishlist. EF Core maps instances of this class to rows in the Minimal API's `minimal_api_applications` table. In short, this class is both the **shape of an application record** and the **source of its model-level validation rules**.

Program.cs

Byte 1: What Program.cs Is Responsible For

Builds on: None — starting point

In plain terms:

Program.cs is the **startup and configuration file** for this Minimal API. It builds the application, registers services such as EF Core and CORS, configures middleware, maps the API endpoints, optionally creates the database and seed data, and finally starts the server.

The code:

```
var builder = WebApplication.CreateBuilder(args);
```

```
// register services/configuration...
```

```
var app = builder.Build();
```

```
// configure middleware/routes...
```

```
app.Run();
```

What's happening:

There are two major phases:

builder → configure the application

↓

Build()

↓

app → configure and run the application

Why it matters:

Most of the application's infrastructure is wired together here. If something isn't registered or configured here, the rest of the application may not be able to use it.

Byte 2: Handling PostgreSQL DateTime Behavior

Builds on: Byte 1

In plain terms:

This application uses both calendar-style dates and UTC timestamps. The Npgsql

compatibility switch allows different `DateTime` kinds to be handled without the newer strict timestamp behavior causing conflicts.

The code:

```
AppContext.SetSwitch(  
    "Npgsql.EnableLegacyTimestampBehavior",  
    true);
```

What's happening:

`Npgsql` is the PostgreSQL provider used by EF Core. Modern `Npgsql` versions have stricter rules around mapping `C# DateTime` values to PostgreSQL timestamp types.

This application deliberately has values such as:

`AppliedOn` → plain calendar date

`CreatedAt` → UTC timestamp

The switch enables legacy timestamp handling.

Why it matters:

Without compatible timestamp behavior, saving certain `DateTime` values could result in runtime errors due to mismatched `DateTime.Kind` and PostgreSQL timestamp expectations.

Byte 3: Creating the Application Builder

Builds on: Byte 1

In plain terms:

`WebApplication.CreateBuilder()` creates the object used to configure the application before it is built.

The code:

```
var builder = WebApplication.CreateBuilder(args);
```

What's happening:

The builder loads configuration, environment information, logging setup, dependency injection services, and other ASP.NET Core infrastructure.

`args` passes command-line arguments into the application's configuration system.

Why it matters:

This builder becomes the central place where services such as EF Core, Swagger, CORS, and authorization are registered before the actual web application is created.

Byte 4: Registering Minimal API and Swagger Services

Builds on: Byte 3

In plain terms:

This application doesn't use MVC controllers, so it registers the services needed for Minimal APIs and Swagger/OpenAPI documentation.

The code:

```
builder.Services.AddEndpointsApiExplorer();  
builder.Services.AddSwaggerGen();  
builder.Services.AddAuthorization();
```

What's happening:

- AddEndpointsApiExplorer() allows Minimal API endpoints to be discovered for API documentation.
- AddSwaggerGen() generates Swagger/OpenAPI documentation.
- AddAuthorization() registers ASP.NET Core's authorization services.

Why it matters:

Swagger provides an interactive way to inspect/test the API, while authorization services prepare the application for endpoint authorization.

Byte 5: Loading the PostgreSQL Connection String

Builds on: Byte 3

In plain terms:

The application needs a connection string to know where PostgreSQL is and how to connect to it. This code tries configuration sources in a specific order instead of hardcoding the connection immediately.

The code:

```
var connectionString =  
    builder.Configuration.GetConnectionString("Default")  
    ?? Environment.GetEnvironmentVariable(  
        "ConnectionStrings__Default")  
    ??  
    "Host=127.0.0.1;Port=5432;Database=placement_tracker;Username=postgres;Password=postgres";
```

What's happening:

The ?? operator means "use the next option if the previous value is null."

Conceptually:

Configuration → Environment variable → Fallback

The intended configuration key is Default.

Why it matters:

Configuration can change between development, testing, and deployment without changing the source code. The fallback keeps the application runnable locally when no external configuration is supplied.

Byte 6: Registering ApplicationDbContext with Dependency Injection

Builds on: Bytes 3 and 5

In plain terms:

This tells ASP.NET Core how to create ApplicationDbContext and connect it to PostgreSQL.

The code:

```
builder.Services.AddDbContext<ApplicationDbContext>(
    options =>
        options.UseNpgsql(connectionString));
```

What's happening:

AddDbContext() registers the EF Core context in ASP.NET Core's **dependency injection (DI)** container.

UseNpgsql() tells EF Core:

Use PostgreSQL

↓

using this connection string

↓

for ApplicationDbContext

Later, an endpoint can simply request ApplicationDbContext db and ASP.NET Core supplies it.

Why it matters:

This connects the endpoint layer to PostgreSQL without the endpoints having to manually create database connections.

Byte 7: Defining the CORS Policy

Builds on: Byte 3

In plain terms:

The frontend and backend can run on different origins, such as different ports. **CORS (Cross-Origin Resource Sharing)** controls which browser origins are allowed to call this API.

The code:

```
const string VueDevPolicy = "VueDev";

builder.Services.AddCors(options =>
{
    options.AddPolicy(VueDevPolicy, policy =>
    {
        policy.WithOrigins(
            "http://localhost:5174",
            "http://localhost:8081")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});
```

What's happening:

Only these two origins are explicitly allowed to make browser requests:

Vue dev server → localhost:5174

NGINX entry → localhost:8081

The policy also allows any HTTP header and HTTP method from those origins.

Why it matters:

Without appropriate CORS configuration, the browser can block frontend requests even though the API itself is running correctly.

Byte 8: Building the Web Application

Builds on: Bytes 3–7

In plain terms:

After all required services have been registered, `builder.Build()` creates the actual web application.

The code:

```
var app = builder.Build();
```

What's happening:

The builder phase is finished. `app` now represents the configured ASP.NET Core application that can have middleware, routes, and startup behavior attached to it.

Why it matters:

You generally register services before `Build()` and configure the request pipeline after `Build()`.

Byte 9: Enabling Swagger Only in Development

Builds on: Byte 8

In plain terms:

Swagger's interactive UI is enabled only when the application is running in the Development environment.

The code:

```
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}
```

What's happening:

ASP.NET Core knows which environment the application is running under.

When it is Development, the application enables:

Swagger JSON

+

Swagger UI

In other environments, these middleware components aren't added by this block.

Why it matters:

Development tooling doesn't have to be exposed in every deployment environment.

Byte 10: Applying CORS and Authorization Middleware

Builds on: Bytes 7–9

In plain terms:

Registering CORS/authorization services isn't enough; the request pipeline must also use them.

The code:

```
app.UseCors(VueDevPolicy);
app.UseAuthorization();
```

What's happening:

Incoming HTTP request



CORS middleware



Authorization middleware



API endpoint

UseCors() applies the policy created earlier. UseAuthorization() enables authorization processing for requests.

Why it matters:

This is the difference between **registering a service** and **actually putting its behavior into the HTTP request pipeline**.

Byte 11: Mapping the Minimal API Endpoints

Builds on: Byte 8

In plain terms:

This line connects the endpoint definitions from ApplicationEndpoints to the running web application.

The code:

```
app.MapApplicationEndpoints();
```

What's happening:

MapApplicationEndpoints() is the extension method defined in the endpoint class.

It internally registers routes such as:

GET /api/applications

GET /api/applications/{id}

POST /api/applications

PUT /api/applications/{id}

DELETE /api/applications/{id}

Why it matters:

Without this call, the endpoint definitions exist in code but aren't actually mapped into the application's HTTP routing system.

Byte 12: Creating a Dependency Injection Scope

Builds on: Byte 6

In plain terms:

Before performing startup database work, the application creates a temporary dependency-injection scope and obtains an `ApplicationDbContext` from it.

The code:

```
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider
        .GetRequiredService<ApplicationDbContext>();

    // database work
}
```

What's happening:

The application asks its DI container for the same `ApplicationDbContext` type used by the endpoints.

using ensures the temporary scope is disposed after the database initialization work finishes.

Why it matters:

Database contexts are scoped services. Creating a scope gives this startup code a proper lifetime for using the context safely.

Byte 13: Creating the Database if Necessary

Builds on: Byte 12

In plain terms:

The application checks whether the database structure exists and creates it if necessary.

The code:

```
db.Database.EnsureCreated();
```

What's happening:

EF Core checks the database and creates the database schema if it doesn't exist.

Importantly, `EnsureCreated()` is not a migration system. It is a simple schema-initialization mechanism.

Why it matters:

For this project, it makes local setup convenient: starting the application can create the required table without manually running a database initialization command.

Byte 14: Keeping Tests Away From the Real Database

Builds on: Byte 13

In plain terms:

Database initialization is deliberately skipped when the application runs in the Testing environment.

The code:

```
if (!app.Environment.IsEnvironment("Testing"))
{
    // EnsureCreated + seeding
}
```

What's happening:

If the environment is Testing, this entire block is skipped.

That means integration tests can configure their own isolated database/provider instead of accidentally creating or modifying the development PostgreSQL database.

Why it matters:

This is an important safety boundary for automated tests: test execution should not unexpectedly change real development data.

Byte 15: Restricting Seed Data to Development

Builds on: Bytes 13–14

In plain terms:

The application contains three sample applications, but they are inserted only during Development and only when the Minimal API table is empty.

The code:

```
if (app.Environment.IsDevelopment()
    && !db.Applications.Any())
{
    db.Applications.AddRange(
        // three sample Application objects
    );

    db.SaveChanges();
}
```

What's happening:

There are two conditions:

Development environment?

AND

minimal_api_applications is empty?

↓

Seed sample records

Any() checks whether at least one record already exists.

Why it matters:

Developers get useful sample data automatically, while production isn't populated with these sample records.

Byte 16: Why Any() Makes the Seeding Idempotent

Builds on: Byte 15

In plain terms:

The `!db.Applications.Any()` check prevents the same sample records from being inserted every time the application starts.

The code:

```
if (app.Environment.IsDevelopment()
    && !db.Applications.Any())
{
    db.Applications.AddRange(...);
    db.SaveChanges();
}
```

What's happening:

First startup:

Table empty → seed 3 records

Next startup:

Table has records → skip seeding

This behavior is called **idempotent seeding** in this context: repeating application startup doesn't keep duplicating the same initial data.

Why it matters:

Without the `Any()` guard, restarting the application could continuously create duplicate sample records.

Byte 17: The Seed Data Uses the Minimal API Entity

Builds on: Bytes 15–16

In plain terms:

The seed records are explicitly instances of the Minimal API's Application model, so they are saved through the Minimal API's ApplicationDbContext mapping.

The code:

```
new PlacementTracker.MinimalApi.Models.Application
{
    Company = "Acme Labs",
    Role = "Python Intern",
    Status = "Applied",
    AppliedOn = new DateTime(2026, 9, 16),
    JobUrl = "https://example.com/jobs/acme-python-intern",
    Notes = "Sample record.",
    CreatedAt = DateTime.UtcNow
}
```

What's happening:

These objects go through:

Minimal API Application

↓

ApplicationDbContext

↓

minimal_api_applications

They do not use the controller-based application's entity/table.

Why it matters:

This preserves the separation between the two implementations running in the same overall project/database environment.

Byte 18: Starting the Web Server

Builds on: All previous bytes

In plain terms:

After configuration, middleware, routes, database setup, and optional seeding are complete, `app.Run()` starts the web application.

The code:

```
app.Run();
```

What's happening:

The application begins listening for HTTP requests according to its configured hosting settings.

From this point, requests can reach routes such as:

GET `/api/applications`

POST `/api/applications`

PUT `/api/applications/1`

DELETE `/api/applications/1`

Why it matters:

Without `app.Run()`, the application would finish executing the startup code without actually running as a web server.

Byte 19: Making Program Discoverable to Integration Tests

Builds on: Byte 18

In plain terms:

The final declaration makes the generated `Program` class accessible to integration-test infrastructure such as `WebApplicationFactory`.

The code:

```
public partial class Program { }
```

What's happening:

Modern top-level C# statements generate the `Program` class implicitly. This declaration makes that generated class accessible in a way test projects can reference.

Integration tests can then use the application itself rather than manually starting a separate server.

Why it matters:

Without this, test infrastructure using `WebApplicationFactory<Program>` may not be able to locate the application's entry point.

PUTTING IT TOGETHER

Program.cs wires the entire Minimal API together from startup to execution. First, it creates the builder and registers Swagger, authorization, EF Core/PostgreSQL, and CORS. After Build(), it configures middleware and maps the application CRUD endpoints. It then performs safe database initialization, skipping that work during testing and adding sample data only when running in development with an empty Minimal API table. Finally, app.Run() starts the server, while the partial Program declaration makes the entry point accessible to integration tests.

MinimalApiTests.cs

Byte 1: What MinimalApiTests Is Testing

Builds on: None — starting point

In plain terms:

This test class verifies that the backend is actually built as a **Minimal API** and that its important database and validation behavior works correctly. It tests both the architecture itself and basic CRUD behavior.

The code:

```
public class MinimalApiTests
{
    [Fact]
    public void DbContext_maps_to_minimal_api_table_not_legacy_table()
    {
        // test
    }

    [Fact]
    public void Backend_uses_no_controller_infrastructure()
    {
        // test
    }

    // more tests...
}
```

What's happening:

[Fact] tells xUnit that each method is an independent automated test.

The tests cover:

Architecture → Minimal API, no controllers

Database → correct table mapping

Validation → valid/invalid DTO behavior

CRUD → create, read, update, delete

Why it matters:

These tests don't just check whether individual functions work; they also protect the project's requirement that this backend remains a Minimal API implementation.

Byte 2: Testing the EF Core Table Mapping

Builds on: Byte 1

In plain terms:

The first test verifies that the Application entity maps to `minimal_api_applications`, not the legacy controller-based Applications table.

The code:

```
var entityType =  
    db.Model.FindEntityType(typeof(Models.Application));  
  
Assert.NotNull(entityType);  
  
Assert.Equal(  
    "minimal_api_applications",  
    entityType!.GetTableName());
```

What's happening:

The test creates an `ApplicationDbContext`, asks EF Core for the metadata describing the Application entity, and checks the table name EF Core has configured.

Why it matters:

The two backend implementations intentionally use different tables. This test catches an accidental change that could make the Minimal API read from or write to the legacy table.

Byte 3: Verifying the Central Table Name

Builds on: Byte 2

In plain terms:

The test also checks that the EF Core mapping and the application's shared `TableName` constant agree.

The code:

```
Assert.Equal(  
    "minimal_api_applications",  
    ApplicationDbContext.TableName);
```

What's happening:

There are effectively two things being verified:

EF Core entity mapping

↓

minimal_api_applications

ApplicationDbContext.TableName

↓

minimal_api_applications

Both must point to the same table.

Why it matters:

If the constant or mapping were changed independently, this test could catch that inconsistency.

Byte 4: Proving There Are No Controllers

Builds on: Byte 1

In plain terms:

Because this project is specifically supposed to use Minimal APIs, the test searches the compiled assembly for classes whose names end in Controller.

The code:

```
var assembly = typeof(Program).Assembly;
```

```
var controllerTypes = assembly.GetTypes()
```

```
    .Where(t => t.Name.EndsWith(
```

```
        "Controller",
```

```
        StringComparison.OrdinalIgnoreCase))
```

```
    .ToList();
```

```
Assert.Empty(controllerTypes);
```

What's happening:

Reflection allows the test to inspect types that exist inside the compiled application.

It asks:

"Does this assembly contain any class that looks like a controller?"

Assert.Empty() requires the answer to be no.

Why it matters:

Someone could accidentally add a traditional controller later and violate the intended Minimal API architecture. This test acts as an architectural guard.

Byte 5: Checking Source Code for Controller Infrastructure

Builds on: Byte 4

In plain terms:

Checking compiled controller classes isn't enough, because controller-related code could still exist in source files. This test therefore scans the backend .cs files for controller-specific APIs and attributes.

The code:

```
var sourceFiles = Directory.GetFiles(  
    Path.Combine(  
        AppContext.BaseDirectory,  
        "..", "..", "..", "..", "backend"),  
    "*.cs",  
    SearchOption.AllDirectories);
```

What's happening:

The test locates all C# files underneath the backend directory, including files in subdirectories.

SearchOption.AllDirectories means the search is recursive.

Why it matters:

This lets the test enforce the architecture at the source-code level rather than only checking compiled types.

Byte 6: Removing Comments Before Checking the Code

Builds on: Byte 5

In plain terms:

The source scan ignores // comments before looking for forbidden controller code. This prevents documentation comments from causing false failures.

The code:

```
var code = string.Join("\n",
```

```
text.Split("\n")
    .Select(line =>
        line.Split(
            new[] { "/" },
            StringSplitOptions.None)[0]));
```

What's happening:

For each source line, the test keeps only the part before //.

For example:

```
// ControllerBase is not used
```

becomes effectively:

```
(empty)
```

Why it matters:

The test is checking actual source code constructs, not words that happen to appear inside explanatory comments.

Byte 7: Checking for Forbidden Controller Patterns

Builds on: Byte 6

In plain terms:

The test explicitly rejects common controller infrastructure such as ControllerBase, ApiController, HTTP method attributes, and MapControllers().

The code:

```
Assert.DoesNotContain("ControllerBase", code);
Assert.DoesNotContain("[ApiController]", code);
Assert.DoesNotContain("[HttpGet]", code);
Assert.DoesNotContain("[HttpPost]", code);
Assert.DoesNotContain("[HttpPut]", code);
Assert.DoesNotContain("[HttpDelete]", code);
Assert.DoesNotContain("MapControllers", code);
```

What's happening:

Each assertion searches the cleaned source code for a controller-specific construct.

If even one is found, the test fails.

Why it matters:

This protects the project's architectural requirement: routes should be mapped using Minimal API methods rather than controller attributes.

Byte 8: Confirming the Endpoint File Exists

Builds on: Bytes 4–7

In plain terms:

The next test verifies that the Minimal API endpoint definitions actually exist in the expected `Endpoints/ApplicationEndpoints.cs` file.

The code:

```
var dir = Path.Combine(
    AppContext.BaseDirectory,
    "..", "..", "..", "..",
    "backend", "Endpoints");

var file = Path.Combine(
    dir,
    "ApplicationEndpoints.cs");

Assert.True(
    File.Exists(file),
    "Endpoints/ApplicationEndpoints.cs must exist.");
```

What's happening:

The test constructs the expected file path and checks whether the file exists.

Why it matters:

This makes the expected project structure part of the test contract. If the endpoint file is deleted or moved unexpectedly, the test catches it.

Byte 9: Confirming Minimal API Route Mapping

Builds on: Byte 8

In plain terms:

Finding the endpoint file isn't enough. The test also checks that it actually contains the four Minimal API CRUD mapping methods.

The code:

```
Assert.Contains("MapGet", text);
Assert.Contains("MapPost", text);
Assert.Contains("MapPut", text);
Assert.Contains("MapDelete", text);
```

What's happening:

The test reads the source file as text and searches for:

MapGet

MapPost

MapPut

MapDelete

These correspond to the four main HTTP CRUD operations.

Why it matters:

It provides a simple architectural check that the endpoint implementation really uses Minimal API route mapping.

Byte 10: Testing Valid DTO Validation

Builds on: The validation logic from the DTO/endpoint code

In plain terms:

This test checks the "happy path": when a DTO contains valid data, `ValidateDto()` should return null, meaning there are no validation errors.

The code:

```
var result =
    ApplicationEndpoints.ValidateDto(
        TestHelpers.ValidCreateDto());
```

```
Assert.Null(result);
```

What's happening:

`ValidCreateDto()` creates a known-good test object.

Then:

Valid DTO

↓

`ValidateDto()`

↓

null

Why it matters:

It confirms that the validation system doesn't reject legitimate application data.

Byte 11: Testing Invalid DTO Validation

Builds on: Byte 10

In plain terms:

This test deliberately makes a valid DTO invalid by removing the company and supplying an unsupported status.

The code:

```
var dto = TestHelpers.ValidCreateDto();

dto.Company = "";
dto.Status = "Hired";

var result =
    ApplicationEndpoints.ValidateDto(dto);

Assert.NotNull(result);
```

What's happening:

Two validation rules are violated:

Company = "" → invalid

Status = "Hired" → invalid

The important expectation is that `ValidateDto()` returns an HTTP result instead of null.

Why it matters:

This verifies that the endpoint validation helper actually catches invalid client input.

Byte 12: Executing the Validation Result as an HTTP Response

Builds on: Byte 11

In plain terms:

The test goes one step further: it actually executes the returned Minimal API result and checks the HTTP status code.

The code:

```
var httpContext = new DefaultHttpContext();
```

```
httpContext.RequestServices =  
    new ServiceCollection()  
        .AddLogging()  
        .BuildServiceProvider();
```

```
await result!.ExecuteAsync(httpContext);
```

```
Assert.Equal(  
    StatusCodes.Status400BadRequest,  
    httpContext.Response.StatusCode);
```

What's happening:

ValidationProblem() returns an IActionResult. The test creates a fake HTTP context and asks the result to execute against it.

The expected result is:

Validation failure



HTTP 400 Bad Request

Why it matters:

The test isn't merely checking that "some error exists"; it verifies the actual HTTP behavior the frontend will receive.

Byte 13: Creating an Isolated Database for CRUD Testing

Builds on: Bytes 1–12

In plain terms:

The CRUD test uses TestHelpers.CreateDb() to obtain a test database context. This keeps the CRUD test separate from the real PostgreSQL database.

The code:

```
using var db = TestHelpers.CreateDb();
```

What's happening:

The helper creates a database context configured specifically for testing.

The test can then perform:

Create → Read → Update → Delete

without depending on real application data.

Why it matters:

Tests should be repeatable and isolated. A test shouldn't depend on whatever records happen to exist in the developer's PostgreSQL database.

Byte 14: Testing Create

Builds on: Byte 13

In plain terms:

The first CRUD operation tested is creating an application and saving it.

The code:

```
var app = TestHelpers.ValidApplication();
```

```
db.Applications.Add(app);
```

```
await db.SaveChangesAsync();
```

```
Assert.Equal(1, db.Applications.Count());
```

What's happening:

A valid entity is added to the test database. `SaveChangesAsync()` persists it, and the test verifies that exactly one application now exists.

Why it matters:

This confirms that EF Core can successfully persist an application using the test configuration.

Byte 15: Testing Read and Ordering

Builds on: Byte 14

In plain terms:

After creating the application, the test reads it back using the same descending-ID ordering used by the real GET-all endpoint.

The code:

```
var all = await db.Applications
    .OrderByDescending(a => a.Id)
    .ToListAsync();
```

```
Assert.Single(all);
```

What's happening:

The test executes a database query that matches the endpoint's ordering behavior.

Because only one record exists, `Assert.Single()` confirms exactly one result was returned.

Why it matters:

It verifies the read side of the CRUD flow and keeps the test aligned with the actual query behavior of the API.

Byte 16: Testing Update

Builds on: Byte 15

In plain terms:

The test retrieves the saved application, changes its company, saves it, and checks that the change persisted.

The code:

```
var existing =  
    await db.Applications.FindAsync(app.Id);
```

```
Assert.NotNull(existing);
```

```
existing!.Company = "Contoso";
```

```
await db.SaveChangesAsync();
```

```
Assert.Equal(  
    "Contoso",  
    db.Applications.Single().Company);
```

What's happening:

EF Core retrieves the existing entity and tracks it. Changing `Company` marks the entity as modified, and `SaveChangesAsync()` persists the update.

Why it matters:

This confirms that EF Core's normal tracked-entity update mechanism works correctly.

Byte 17: Testing Delete

Builds on: Byte 16

In plain terms:

The final CRUD operation removes the existing application and verifies that nothing remains.

The code:

```
db.Applications.Remove(existing);  
await db.SaveChangesAsync();
```

```
Assert.Empty(db.Applications.ToList());
```

What's happening:

Remove() marks the entity for deletion. SaveChangesAsync() sends the deletion to the database.

The final assertion confirms:

Before → 1 application

Delete → Save

After → 0 applications

Why it matters:

Together with the previous CRUD tests, this verifies the complete database lifecycle of an application.

Byte 18: What This Test Suite Protects

Builds on: All previous bytes

In plain terms:

These tests protect both **behavior** and **architecture**. They verify not only that applications can be stored and validated, but also that the project remains a Minimal API and continues using the correct database table.

The code:

```
// Architecture
```

```
Backend_uses_no_controller_infrastructure()
```

```
// Routing
```

```
Endpoints_file_uses_minimal_api_mapping()
```

```
// Database mapping
```

```
DbContext_maps_to_minimal_api_table_not_legacy_table()
```

```
// Validation
```

```
ValidateDto_valid_returns_null()
```

```
ValidateDto_invalid_returns_400_validation_problem()
```

```
// Data operations
```

```
InMemory_crud_roundtrip_against_new_context()
```

What's happening:

The tests cover several layers:

Architecture

↓

Endpoint structure

↓

Validation

↓

EF Core database behavior

↓

CRUD lifecycle

Why it matters:

A future developer can change the implementation while these tests act as guardrails against accidentally breaking the project's required behavior or architecture.

PUTTING IT TOGETHER

This test class is doing more than checking whether CRUD works. First, it verifies that the application is genuinely using **Minimal API architecture** and has no controller infrastructure. It then checks that the Application entity maps specifically to `minimal_api_applications`, validates that the custom DTO validation produces the expected HTTP 400 response, and finally exercises the database through create, read, update, and delete operations. The result is a combination of **architectural tests, validation tests, and data-access tests**. If all of these pass, the code has evidence that both the intended design and the important runtime behaviors are still intact.

IntegrationTests.cs

Byte 1: What IntegrationTests Is Testing

Builds on: None — starting point

In plain terms:

These are **integration tests**, meaning they test the actual API through HTTP instead of calling endpoint methods directly. The tests verify that routing, JSON serialization, validation, EF Core, and HTTP responses work together as one system.

The code:

```
public class IntegrationTests
    : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly WebApplicationFactory<Program> _factory;
}
```

What's happening:

WebApplicationFactory<Program> creates a test version of the ASP.NET Core application. The tests can then create an HttpClient and send requests such as a real frontend would.

Why it matters:

A unit test might verify one method in isolation. These tests verify that the **whole HTTP pipeline** actually works.

Byte 2: Reusing the Test Application

Builds on: Byte 1

In plain terms:

IClassFixture allows xUnit to share one configured WebApplicationFactory across the tests in this class.

The code:

```
public class IntegrationTests
    : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly WebApplicationFactory<Program> _factory;

    public IntegrationTests(
        WebApplicationFactory<Program> factory)
```

```

{
    // configure factory
}
}

```

What's happening:

xUnit provides the factory to the test class constructor. The constructor then creates a customized version of that factory and stores it in `_factory`.

Why it matters:

Every test can use the same application setup without manually starting the ASP.NET Core server each time.

Byte 3: Creating an Isolated Test Database Name

Builds on: Byte 2

In plain terms:

The tests use an EF Core InMemory database instead of the real PostgreSQL database. A unique name is generated so the test database is isolated.

The code:

```

var dbName =
    "MinimalApi-IntegrationTests-" + Guid.NewGuid();

```

What's happening:

`Guid.NewGuid()` generates a unique identifier, producing something conceptually like:

MinimalApi-IntegrationTests-a3f2...

This name is created **before** configuring the services.

Why it matters:

The tests shouldn't accidentally read or modify your real `placement_tracker` PostgreSQL database.

Byte 4: Why the Database Name Is Created Outside ConfigureServices

Builds on: Byte 3

In plain terms:

The comment here addresses an important testing gotcha: `ConfigureServices` may execute more than once. If the `Guid` were generated inside it, different executions could receive different database names.

The code:

```

var dbName = "MinimalApi-IntegrationTests-" + Guid.NewGuid();

```

```

_factory = factory.WithWebHostBuilder(builder =>
{
    builder.ConfigureServices(services =>
    {
        services.AddDbContext<ApplicationDbContext>(
            options => options.UseInMemoryDatabase(dbName));
    });
});

```

What's happening:

One database name is generated and then captured by the configuration delegate.

One factory setup

↓

One dbName

↓

All configured DbContexts use same InMemory store

Why it matters:

If different contexts accidentally used different InMemory stores, a POST in one scope might not be visible to a subsequent GET.

Byte 5: Running the Application in Testing

Builds on: Bytes 1–4

In plain terms:

The test application is explicitly given the Testing environment so Program.cs knows it is running under tests.

The code:

```
builder.UseEnvironment("Testing");
```

What's happening:

Earlier, Program.cs checks:

```

if (!app.Environment.IsEnvironment("Testing"))
{
    // database initialization

```

```
}
```

Therefore, the integration test environment skips the real PostgreSQL initialization and seed-data logic.

Why it matters:

This creates a clear boundary:

Production/Development → normal database behavior

Testing → isolated test database

Byte 6: Replacing the Real EF Core Database Configuration

Builds on: Bytes 3–5

In plain terms:

The application normally registers `ApplicationDbContext` with PostgreSQL. The integration test removes that registration and replaces it with an `InMemory` database.

The code:

```
var descriptor = services.SingleOrDefault(
    d => d.ServiceType ==
        typeof(DbContextOptions<ApplicationDbContext>));
```

```
if (descriptor != null)
{
    services.Remove(descriptor);
}
```

```
services.AddDbContext<ApplicationDbContext>(
    options => options.UseInMemoryDatabase(dbName));
```

What's happening:

The test first finds the existing EF Core configuration and removes it.

Then it registers a new configuration:

`ApplicationDbContext`

↓

`InMemory` database

instead of:

ApplicationDbContext



PostgreSQL

Why it matters:

The API code doesn't need to know it is being tested. It still requests ApplicationDbContext; only the underlying database provider changes.

Byte 7: Creating an HttpClient

Builds on: Bytes 1–6

In plain terms:

Each test creates an HttpClient connected to the in-memory test version of the API.

The code:

```
var client = _factory.CreateClient();
```

What's happening:

This client can make requests such as:

```
await client.GetAsync("/api/applications");
```

without requiring you to manually start a separate web server.

The request still travels through the application's routing and middleware pipeline.

Why it matters:

This makes the test behave much more like a real frontend calling the backend.

Byte 8: Testing GET-All Through HTTP

Builds on: Byte 7

In plain terms:

This test checks that GET /api/applications works through the complete HTTP pipeline and returns a successful response containing JSON.

The code:

```
var response =
```

```
    await client.GetAsync("/api/applications");
```

```
Assert.Equal(
```

```
    HttpStatusCode.OK,
```

```
    response.StatusCode);
```

```
var list =  
    await response.Content  
        .ReadFromJsonAsync<  
            List<DTOs.ApplicationResponseDto>>();
```

```
Assert.NotNull(list);
```

What's happening:

The test verifies two things:

HTTP status → 200 OK

Response → can be deserialized into a list

Why it matters:

It confirms both the endpoint and the JSON response format are working.

Byte 9: Testing GET for a Missing Application

Builds on: Byte 8

In plain terms:

This test checks the API's not-found behavior when the requested ID doesn't exist.

The code:

```
var response =  
    await client.GetAsync("/api/applications/999999");
```

```
Assert.Equal(  
    HttpStatusCode.NotFound,  
    response.StatusCode);
```

What's happening:

The endpoint searches for application 999999. Since the test database doesn't contain that record, the expected response is:

GET /api/applications/999999

↓

404 Not Found

Why it matters:

It verifies that missing resources are handled correctly rather than producing an incorrect 200 OK or an unexpected server error.

Byte 10: Testing the Full CRUD Journey

Builds on: Bytes 7–9

In plain terms:

The largest test performs a complete application lifecycle through HTTP: **POST** → **GET** → **PUT** → **DELETE** → **GET**.

The code:

```
// POST

var post = await client.PostAsJsonAsync(
    "/api/applications", new { /* data */ });

// GET

var get = await client.GetAsync(
    $"/api/applications/{created.Id}");

// PUT

var put = await client.PutAsJsonAsync(
    $"/api/applications/{created.Id}", new { /* data */ });

// DELETE

var delete = await client.DeleteAsync(
    $"/api/applications/{created.Id}");

// GET again

var getAfter = await client.GetAsync(
    $"/api/applications/{created.Id}");
```

What's happening:

The test uses the ID returned by POST for all subsequent operations.

The expected lifecycle is:

POST → 201 Created

GET → 200 OK

PUT → 200 OK

DELETE → 204 No Content

GET → 404 Not Found

Why it matters:

This tests whether the different endpoints work together, not merely whether each endpoint works independently.

Byte 11: Sending JSON with PostAsJsonAsync

Builds on: Byte 10

In plain terms:

PostAsJsonAsync() makes the test request look like a real frontend request by converting the supplied C# object into JSON.

The code:

```
var post = await client.PostAsJsonAsync(
    "/api/applications",
    new
    {
        company = "Acme Labs",
        role = "Backend Intern",
        status = "Applied",
        appliedOn = "2026-09-16",
        jobUrl = "https://example.com/job/123",
        notes = "Integration test"
    });
```

What's happening:

The anonymous C# object is serialized into JSON and sent as the HTTP request body.

The API then binds that JSON to CreateApplicationDto.

Why it matters:

This tests the actual boundary between frontend-style JSON input and the backend's DTO model.

Byte 12: Verifying POST Creates the Correct Response

Builds on: Byte 11

In plain terms:

After POST, the test checks both the HTTP status and the returned application data.

The code:

```
Assert.Equal(  
    HttpStatusCode.Created,  
    post.StatusCode);  
  
var created =  
    await post.Content.ReadFromJsonAsync<  
        DTOs.ApplicationResponseDto>();  
  
Assert.NotNull(created);  
Assert.Equal(  
    "Acme Labs",  
    created!.Company);
```

What's happening:

The response is expected to be:

```
201 Created  
  
+  
  
ApplicationResponseDto
```

The returned Id is then used by later operations.

Why it matters:

It proves that creation didn't just return the correct status; it actually returned the newly created resource.

Byte 13: Testing PUT Through HTTP

Builds on: Byte 12

In plain terms:

The test updates the application using its newly created ID and expects 200 OK.

The code:

```
var put = await client.PutAsJsonAsync(
    $"/api/applications/{created.Id}",
    new
    {
        company = "Contoso",
        role = "Offer Role",
        status = "Offer",
        appliedOn = "2026-09-05",
        jobUrl = "https://example.com/offer",
        notes = "Updated"
    });
```

```
Assert.Equal(
    HttpStatusCode.OK,
    put.StatusCode);
```

What's happening:

The JSON body is bound to UpdateApplicationDto, validated, applied to the existing entity, and saved.

Why it matters:

This confirms the entire update path works from HTTP request to database update.

Byte 14: Testing DELETE and Confirming Removal

Builds on: Byte 13

In plain terms:

The test deletes the application and then performs another GET to prove that the record is actually gone.

The code:

```
var delete =
    await client.DeleteAsync(
        $"/api/applications/{created.Id}");
```

```
Assert.Equal(
```

```
        HttpStatusCode.NoContent,  
        delete.StatusCode);  
  
var getAfter =  
    await client.GetAsync(  
        $"/api/applications/{created.Id}");  
  
Assert.Equal(  
    HttpStatusCode.NotFound,  
    getAfter.StatusCode);
```

What's happening:

The test doesn't stop at verifying 204.

It verifies the database state indirectly:

DELETE → 204

GET same ID → 404

Why it matters:

This proves the DELETE operation actually removed the resource rather than simply returning a successful status.

Byte 15: Testing Invalid POST Requests

Builds on: Bytes 10–12

In plain terms:

This test sends deliberately invalid application data and verifies that the API rejects it with 400 Bad Request.

The code:

```
var response = await client.PostAsJsonAsync(  
    "/api/applications",  
    new  
    {  
        company = "",  
        role = "x",  
        status = "Hired",
```

```
        appliedOn = (string?)null
    });
```

```
Assert.Equal(
    HttpStatusCode.BadRequest,
    response.StatusCode);
```

What's happening:

Several validation rules are violated:

Company = "" → invalid

Role = "x" → too short

Status = "Hired" → not allowed

AppliedOn = null → invalid for this status

The expected result is HTTP 400.

Why it matters:

This confirms validation works through the real HTTP pipeline, not just when `ValidateDto()` is called directly.

Byte 16: Testing PUT for a Missing Application

Builds on: Byte 13

In plain terms:

This test sends a valid update request but targets an ID that doesn't exist. The API should return 404, not 400.

The code:

```
var response = await client.PutAsJsonAsync(
    "/api/applications/999999",
    new
    {
        company = "Contoso",
        role = "Backend Intern",
        status = "Applied",
        appliedOn = "2026-09-05"
    });
```

```
Assert.Equal(  
    HttpStatusCode.NotFound,  
    response.StatusCode);
```

What's happening:

The request data itself is valid. The problem is the resource doesn't exist.

So:

Valid DTO + missing ID → 404

Why it matters:

This distinguishes **invalid input** from **missing resources**, which are different API conditions.

Byte 17: Testing DELETE for a Missing Application

Builds on: Byte 14

In plain terms:

The final test checks that attempting to delete an application that doesn't exist returns 404 Not Found.

The code:

```
var response =  
    await client.DeleteAsync(  
        "/api/applications/999999");
```

```
Assert.Equal(  
    HttpStatusCode.NotFound,  
    response.StatusCode);
```

What's happening:

The endpoint searches for ID 999999. Since it doesn't exist, it doesn't attempt a deletion and returns 404.

Why it matters:

This verifies consistent not-found behavior across the CRUD endpoints.

Byte 18: What Makes These Integration Tests Different

Builds on: All previous bytes

In plain terms:

These tests don't call `ApplicationEndpoints` methods directly. They send real HTTP requests to a test instance of the application, while swapping PostgreSQL for an isolated InMemory database.

The code:

Test



HttpClient



ASP.NET Core pipeline



Minimal API endpoint



ApplicationDbContext



InMemory database

What's happening:

The test environment replaces only the database implementation. The actual routes, DTO binding, validation, endpoint logic, response generation, and middleware still participate in the request.

Why it matters:

This catches integration problems that isolated unit tests might miss—for example, incorrect routing, JSON binding problems, wrong HTTP status codes, or endpoint-to-database wiring issues.

PUTTING IT TOGETHER

`IntegrationTests` creates a complete test version of the Minimal API using `WebApplicationFactory`. It switches the application into the Testing environment and replaces PostgreSQL with a uniquely named InMemory database, keeping tests isolated from real data. Each test then uses `HttpClient` just like a frontend would, sending real HTTP requests through the API pipeline. The suite checks normal CRUD behavior as well as invalid requests and missing resources, verifying expected status codes such as 201, 200, 204, 400, and 404. In short, these tests answer the question: **"Does the entire Minimal API actually behave correctly when used over HTTP?"**

TestHelpers.cs

Byte 1: Purpose of TestHelpers

Builds on: None — starting point

In plain terms:

TestHelpers is a utility class for the test project. It keeps commonly needed test setup and sample data in one place, so individual tests don't have to repeatedly write the same code.

The code:

```
public static class TestHelpers
{
    public static ApplicationDbContext CreateDb()
    {
        // ...
    }

    public static Models.Application ValidApplication() => new()
    {
        // ...
    };
}
```

What's happening:

The class contains reusable helper methods specifically for tests. Here, there are three jobs: create an isolated test database, create a valid Application entity, and create valid Create/Update DTOs.

Why it matters:

Without this helper class, every test would contain duplicated setup code, making tests longer and harder to maintain.

Byte 2: Creating an Isolated Test Database

Builds on: Byte 1

In plain terms:

CreateDb() creates an Entity Framework Core database that exists only for a particular test. It uses EF Core's **InMemory provider**, so the tests don't need to connect to the real PostgreSQL database.

The code:

```

public static ApplicationDbContext CreateDb()
{
    var options = new DbContextOptionsBuilder<ApplicationDbContext>()
        .UseInMemoryDatabase(databaseName: Guid.NewGuid().ToString())
        .Options;

    var db = new ApplicationDbContext(options);
    db.Database.EnsureCreated();
    return db;
}

```

What's happening:

DbContextOptionsBuilder configures how the ApplicationDbContext should access data. UseInMemoryDatabase() tells EF Core to store test data in memory, while Guid.NewGuid() gives the database a unique name each time.

Why it matters:

Tests can safely create, modify, and delete records without touching the real PostgreSQL database or interfering with other tests.

Byte 3: Why the Database Gets a Unique Name

Builds on: Byte 2

In plain terms:

Each call to CreateDb() gets a different database name. A **GUID** is a practically unique identifier, so two tests won't accidentally share the same in-memory database.

The code:

```

.UseInMemoryDatabase(
    databaseName: Guid.NewGuid().ToString()
)

```

What's happening:

Suppose one test gets database name:

8a31...

and another gets:

f42c...

EF Core treats them as separate in-memory databases.

Why it matters:

Test isolation is important. If Test A inserts an application and Test B sees that same application, Test B could pass or fail because of Test A's data rather than because of its own logic.

Byte 4: Returning the Configured DbContext

Builds on: Byte 2

In plain terms:

After configuring the in-memory database, the helper creates the actual `ApplicationDbContext` and returns it to the test.

The code:

```
var db = new ApplicationDbContext(options);  
db.Database.EnsureCreated();  
  
return db;
```

What's happening:

The options tell the context to use the unique in-memory database. `EnsureCreated()` makes sure the database structure is ready before the test starts using it.

A test can then simply do:

```
using var db = TestHelpers.CreateDb();
```

```
db.Applications.Add(...);  
await db.SaveChangesAsync();
```

Why it matters:

The test doesn't need to understand all the database configuration. `CreateDb()` hides that setup behind one simple method.

Byte 5: Creating a Valid Entity Fixture

Builds on: Byte 1

In plain terms:

`ValidApplication()` creates a complete `Models.Application` object containing values that should pass the application's validation rules. This is called **test data** or a **fixture**.

The code:

```
public static Models.Application ValidApplication() => new()  
{
```

```
Company = "Acme Labs",  
Role = "Backend Intern",  
Status = "Applied",  
AppliedOn = new DateTime(2026, 9, 16),  
JobUrl = "https://example.com/job/123",  
Notes = "Test notes",  
CreatedAt = DateTime.UtcNow  
};
```

What's happening:

Instead of every test manually constructing an application, the test can call:

```
var app = TestHelpers.ValidApplication();
```

The returned object has a realistic company, role, status, date, URL, notes, and creation timestamp.

Why it matters:

It gives tests a known-good starting point. If a test is supposed to test CRUD rather than validation, it doesn't need to spend time constructing valid input.

Byte 6: Creating Valid Create DTO Data

Builds on: Byte 5

In plain terms:

This helper creates a valid `CreateApplicationDto`, which represents the data a client would send when creating an application.

The code:

```
public static DTOs.CreateApplicationDto ValidCreateDto() => new()  
{  
    Company = "Acme Labs",  
    Role = "Backend Intern",  
    Status = "Applied",  
    AppliedOn = new DateTime(2026, 9, 16),  
    JobUrl = "https://example.com/job/123",  
    Notes = "Test notes"  
};
```

What's happening:

This is deliberately a **DTO**, not the database entity. Tests such as:

```
var result =
```

```
    ApplicationEndpoints.ValidateDto(  
        TestHelpers.ValidCreateDto());
```

can use it to verify that valid input passes validation.

Why it matters:

It separates testing API input validation from testing database entities. The test can say, essentially, "Give the API a normal valid request and make sure validation accepts it."

Byte 7: Creating Valid Update DTO Data

Builds on: Byte 6

In plain terms:

`ValidUpdateDto()` provides valid data representing an update to an existing application. It uses different values from the create DTO so tests can clearly verify that an update actually changed the record.

The code:

```
public static DTOs.UpdateApplicationDto ValidUpdateDto() => new()  
{  
    Company = "Contoso",  
    Role = "Offer Role",  
    Status = "Offer",  
    AppliedOn = new DateTime(2026, 9, 5),  
    JobUrl = "https://example.com/offer",  
    Notes = "Updated notes"  
};
```

What's happening:

The original test data might say:

Acme Labs → Backend Intern → Applied

while the update data says:

Contoso → Offer Role → Offer

This makes it easy for a test to verify that the update operation really took effect.

Why it matters:

Using clearly different values prevents a test from accidentally passing without actually checking whether the update worked.

PUTTING IT TOGETHER

TestHelpers is the shared setup layer for the Minimal API tests. CreateDb() gives each test a fresh in-memory EF Core database instead of using PostgreSQL. ValidApplication() supplies a valid database entity, while ValidCreateDto() and ValidUpdateDto() supply valid API input objects. The actual tests can therefore focus on **what they are testing**—validation, CRUD, or endpoint behavior—without repeating setup code. In short: **TestHelpers prepares the test environment and standard test data; the test files use those prepared pieces to verify the application.**

ValidationTests.cs

Byte 1: Purpose of ValidationTests

Builds on: None — starting point

In plain terms:

ValidationTests checks that the application's input rules actually work. Instead of testing whether the API can save data, these tests ask: **“Does valid data pass, and does invalid data get rejected?”**

The code:

```
public class ValidationTests
{
    // validation tests...
}
```

What's happening:

The class contains a collection of focused tests covering company, role, status, dates, URLs, and notes.

Why it matters:

Validation protects the application from bad input. These tests make sure those rules don't accidentally break when the code changes.

Byte 2: Creating a Reusable Validation Helper

Builds on: Byte 1

In plain terms:

All the tests need to run the same .NET validation process. Instead of repeating that process in every test, the Validate() helper does it once.

The code:

```
private static IList<ValidationResult> Validate(object app)
{
    var mockServices = new Mock<IServiceProvider>();
    mockServices
        .Setup(s => s.GetService(It.IsAny<Type>()))
        .Returns((object?)null);

    var context = new ValidationContext(
```

```

        app,
        mockServices.Object,
        null);

var results = new List<ValidationResult>();

Validator.TryValidateObject(
    app,
    context,
    results,
    validateAllProperties: true);

return results;
}

```

What's happening:

The helper receives any object, runs its Data Annotation validation, collects any ValidationResult errors, and returns them. An empty list means validation passed.

Why it matters:

Every test can simply call Validate(app) and focus on the specific rule it wants to verify.

Byte 3: Supplying an IServiceProvider with Moq

Builds on: Byte 2

In plain terms:

ValidationContext can accept an IServiceProvider, which is normally used when validation needs access to application services. These tests don't need real services, so **Moq** creates a lightweight fake.

The code:

```

var mockServices = new Mock<IServiceProvider>();

mockServices
    .Setup(s => s.GetService(It.IsAny<Type>()))
    .Returns((object?)null);

```

```
var context = new ValidationContext(  
    app,  
    mockServices.Object,  
    null);
```

What's happening:

Moq creates an object that behaves like `IServiceProvider`. If validation asks for any service, the mock simply returns null.

Why it matters:

The test gets a valid `ValidationContext` without starting the entire application's dependency-injection system.

Byte 4: Running Data Annotation Validation

Builds on: Byte 2

In plain terms:

`Validator.TryValidateObject()` is the .NET mechanism that checks attributes such as `[Required]`, `[StringLength]`, and `[Url]`.

The code:

```
var results = new List<ValidationResult>();
```

```
Validator.TryValidateObject(  
    app,  
    context,  
    results,  
    validateAllProperties: true);
```

```
return results;
```

What's happening:

.NET examines the object's validation attributes and puts any failures into results.

For example:

```
[StringLength(100, MinimumLength = 2)]
```

```
public string Company { get; set; }
```

means values outside that range can produce a `ValidationResult`.

Why it matters:

This gives the tests a direct way to verify the validation rules defined on the model and DTOs.

Byte 5: Verifying Valid Data

Builds on: Byte 4

In plain terms:

A good validation test doesn't only test failures. It also confirms that legitimate data is accepted.

The code:

[Fact]

```
public void Valid_application_passes_validation()
{
    Assert.Empty(
        Validate(TestHelpers.ValidApplication()));
}
```

What's happening:

TestHelpers.ValidApplication() creates known-good data. Validate() runs the rules, and Assert.Empty() confirms there are **zero validation errors**.

The same idea is used for the Create DTO:

```
Assert.Empty(
    Validate(TestHelpers.ValidCreateDto()));
```

Why it matters:

If valid data suddenly starts failing validation, this test catches an unintended regression.

Byte 6: Testing the Wishlist Date Exception

Builds on: Byte 5

In plain terms:

The application has a business rule: an application can be in Wishlist status without having an AppliedOn date. This test verifies that exception.

The code:

```
var app = TestHelpers.ValidApplication();
```

```
app.Status = "Wishlist";
```

```
app.AppliedOn = null;
```

```
Assert.Empty(Validate(app));
```

What's happening:

The test starts with valid data, changes the status to Wishlist, removes the applied date, and expects validation to still succeed.

Why it matters:

This checks a **cross-field rule**: whether AppliedOn is required depends on the value of Status.

Byte 7: Testing Required Company and Role Values

Builds on: Byte 5

In plain terms:

Company and role must contain meaningful text. These tests make sure empty strings and whitespace-only values are rejected.

The code:

[Fact]

```
public void Blank_company_fails()
{
    var app = TestHelpers.ValidApplication();
    app.Company = "";
```

```
    Assert.NotEmpty(Validate(app));
}
```

[Fact]

```
public void Blank_role_fails()
{
    var app = TestHelpers.ValidApplication();
    app.Role = "  ";
```

```
    Assert.NotEmpty(Validate(app));
}
```

What's happening:

The first test uses an empty string. The second deliberately uses spaces to check that “blank” isn't treated as valid just because the string technically contains characters.

Why it matters:

Without this kind of validation, meaningless values such as " " could enter the database.

Byte 8: Testing Minimum and Maximum Lengths

Builds on: Byte 7

In plain terms:

The model requires company and role names to be between **2 and 100 characters**. These tests check both sides of that boundary.

The code:

```
app.Company = "A";
Assert.NotEmpty(Validate(app));

app.Role = "x";
Assert.NotEmpty(Validate(app));
app.Company = new string('a', 101);
Assert.NotEmpty(Validate(app));
```

```
app.Role = new string('b', 101);
Assert.NotEmpty(Validate(app));
```

What's happening:

"A" and "x" test values that are too short. `new string('a', 101)` creates a 101-character string to test the upper limit.

Why it matters:

Boundary tests are useful because off-by-one mistakes are common—for example, accidentally allowing 101 characters.

Byte 9: Testing the Notes Limit

Builds on: Byte 8

In plain terms:

Notes are optional, but if provided they cannot exceed 1,000 characters.

The code:

```
var app = TestHelpers.ValidApplication();
```

```
app.Notes = new string('n', 1001);
```

```
Assert.NotEmpty(Validate(app));
```

What's happening:

The test creates exactly 1,001 characters and verifies that validation rejects them.

Why it matters:

It protects the documented maximum length and prevents unexpectedly large notes from being accepted.

Byte 10: Testing Allowed Status Values

Builds on: Byte 4

In plain terms:

An application status must come from the predefined set, such as Wishlist, Applied, Interview, Offer, or Rejected. This test makes sure an unknown status is rejected.

The code:

```
var app = TestHelpers.ValidApplication();
```

```
app.Status = "Hired";
```

```
Assert.NotEmpty(Validate(app));
```

What's happening:

"Hired" is not one of the allowed statuses, so the model's custom Validate() method should produce an error.

Why it matters:

Without this rule, inconsistent status values could enter the database and break filtering, reporting, or frontend logic.

Byte 11: Testing the Conditional AppliedOn Rule

Builds on: Byte 6

In plain terms:

For every status other than Wishlist, the application must have an AppliedOn date. This test checks the failure case.

The code:

```
var app = TestHelpers.ValidApplication();

app.Status = "Applied";
app.AppliedOn = null;

var errors = Validate(app);

Assert.Contains(
    errors,
    e => e.MemberNames.Contains("AppliedOn"));
```

What's happening:

The status says the user has applied, but there is no application date. The test doesn't merely check that *some* error exists—it specifically checks that the error belongs to AppliedOn.

Why it matters:

This verifies both the business rule and that the validation error is correctly associated with the affected field.

Byte 12: Testing URL Validation

Builds on: Byte 4

In plain terms:

JobUrl is optional, but when a URL is supplied, it must have a valid URL format.

The code:

```
var app = TestHelpers.ValidApplication();

app.JobUrl = "not-a-url";
```

```
Assert.NotEmpty(Validate(app));
```

What's happening:

The [Url] validation attribute examines the supplied value and should produce a validation error because "not-a-url" isn't a valid URL.

Why it matters:

It prevents malformed job links from being stored and later displayed as broken links.

Byte 13: Testing Optional Fields

Builds on: Byte 12

In plain terms:

Not every application needs a job URL or notes. This test confirms that leaving those optional fields empty is allowed.

The code:

```
var app = TestHelpers.ValidApplication();
```

```
app.JobUrl = null;
```

```
app.Notes = null;
```

```
Assert.Empty(Validate(app));
```

What's happening:

The test removes both optional values and expects no validation errors.

Why it matters:

Validation should reject invalid values without accidentally making optional information mandatory.

PUTTING IT TOGETHER

ValidationTests provides a safety net around the application's input rules. The Validate() helper creates the validation context and runs .NET's validation system, while each [Fact] focuses on one specific rule. The tests cover both **valid cases** and **invalid boundary cases**: required fields, length limits, allowed statuses, conditional dates, URLs, and optional fields. Together, they verify that the rules defined in the model/DTOs behave as intended and help catch regressions if those rules are changed later.

